PROJECT PROPOSAL

Corridor Hill Residence Maintenance System

A digital platform for reporting, tracking and managing residence maintenance

Prepared for	Corridor Hill Residence Management
Prepared by	T.Mokhabukhi and MD.Nwobodo
Purpose	Residence maintenance app/web challange
Date	29 October 2026

Proposal at a Glance

Challenge: Maintenance reports can be difficult to consolidate, track and communicate consistently.

Proposal: A central, mobile-friendly reporting and management platform with resident tracking.

Current project: Website, API and database architecture are present in the repository; a companion mobile app is also included, with some integration still in progress.

Recommended next step: Validate workflows, test security and deployment, then run a controlled pilot.

Important implementation note: This document distinguishes features and architecture described in the supplied project from tasks that still require verification. It does not claim that the production service or mobile app is already fully deployed and ready for residence-wide use.

1. Executive Summary

The Corridor Hill Residence Maintenance System is a proposed digital platform that makes it easier for residents to report maintenance problems and for residence management to track, prioritise and communicate progress on those reports.

The supplied project already contains a resident-facing website, a management dashboard, a Node.js/Express API and a React Native/Expo mobile-app codebase. The website supports resident registration and login, accommodation selection, maintenance reporting with photo attachments, complaint tracking, management-side complaint handling, resident/room information and announcements. The mobile app includes resident and management screens, although the project README notes that the mobile data-service migration is still in progress.

This proposal requests approval to validate the system with Corridor Hill Residence management, complete the remaining integration and deployment checks, and run a controlled pilot before wider rollout.

2. Background and Problem Statement

Maintenance problems in a student residence need to be recorded accurately, routed to the right people and followed through until resolution. Paper-based or informal reporting can make it difficult to see which issues have been reported, whether a report has been acknowledged, what work is outstanding and whether a resident has received an update.

Reports can also lack useful evidence or precise location details. When the same issue is reported by several residents, staff may spend time reconciling duplicate reports instead of acting on the problem. A central digital record can help reduce these gaps while preserving management's responsibility to assess and resolve each issue.

3. Proposed Solution

Introduce a mobile-friendly maintenance reporting website, supported by a management dashboard and a planned/partially implemented companion mobile app. Residents can submit a report from a phone or computer, identify their room or unit, describe the problem and attach photographs where useful. Management can review reports in one queue, update their status, add notes and use the recorded information to coordinate the work.

The system is intended to improve visibility and accountability; it does not itself perform maintenance or guarantee a particular repair time. Response times will still depend on staffing, parts, access and residence procedures.

4. Objectives

Make maintenance reporting accessible from residents' phones without requiring a paper form or an in-person visit just to lodge a report.

Capture consistent information about the issue, category, location and supporting photographs.

Give residents a way to review their submitted reports and see the latest recorded status.

Give authorised management staff a central view of open, in-progress and completed requests.

Create a traceable history of status changes, notes and key actions.

Support clearer communication through residence announcements and maintenance updates.

Provide useful operational information to help management identify recurring problems and outstanding work.

5. Main Features

Resident access: account registration and login, with the current website configured to restrict registration to TUT student email addresses ending in @tut4life.ac.za.

Room and bed selection: associate a report with the relevant block, room and unit/bed so staff can locate the problem.

Maintenance reporting: describe the issue, select relevant details and attach supporting photographs within configured upload limits.

Complaint tracking: show the recorded status and timeline of a report, with notes and progress information where available.

Management dashboard: review and manage complaints, residents, rooms/units and announcements through authorised management access.

Prioritisation and scheduling fields: use priority and estimated/actual completion information where configured to help organise work.

Duplicate awareness: residents sharing a unit can see reports already filed for that unit, helping them avoid submitting the same issue repeatedly; editing remains restricted to the report owner or management.

Mobile experience: the repository includes an Expo/React Native application with resident and management navigation and screens. The mobile app's remaining API/data integration and end-to-end behaviour must be tested before it is presented as production-ready.

6. Suggested Maintenance Workflow

1. A resident signs in and selects or confirms their accommodation location.
2. The resident submits the issue, description and optional photos.
3. The report appears in the management queue with its location and recorded details.
4. Management reviews the report, sets priority where appropriate and updates the status as work progresses.
5. Staff record relevant notes and, where possible, an estimated completion date.
6. The resident checks the report timeline and latest status online.
7. Once the work is verified as complete, management marks the report completed. The record is retained or archivedaccording to the system's configured retention policy.

7. Users and Responsibilities

Residents: submit accurate reports, attach appropriate evidence, provide access where required and use the tracking page to check progress.

Residence management: administer authorised access, review and prioritise requests, update statuses, coordinate with maintenance personnel and publish relevant announcements.

Maintenance personnel: receive assigned work through the process agreed with management, carry out repairs and provide completion information for the record.

System administrator/developer: maintain deployment configuration, manage technical updates, monitor errors, apply security fixes and support authorised users.

Management should nominate a system owner and define who is permitted to view resident information, change statuses, close reports and manage user accounts.

8. Technology and Architecture

The supplied repository documents the following architecture: a static HTML5/CSS3/JavaScript website; a Node.js/Express API; a PostgreSQL database; and an Expo/React Native mobile client. The repository includes deployment configuration intended for GitHub Pages for the website and Render for the API/database, while mobile builds are configured around Expo Application Services (EAS). Actual live deployment, hosting availability and production configuration must be confirmed before launch.

The API design uses authenticated requests and role-based access for resident and management functions. The README describes password hashing, expiring JWT access tokens, rate limiting, input limits, restricted photo types and sizes, and production checks for important secrets and CORS configuration. These are useful security foundations, but a security review and live-environment test are still required before real resident data is used.

The project README also states that mobile authentication is already using the API while some mobile data services still use Firebase and are being migrated to the REST API. Completing and testing that migration is a key implementation task.

9. Privacy, Security and Responsible Use

Only collect information needed to process maintenance reports. Restrict resident records and complaint details to authorised users and the residents permitted to view them.

Use HTTPS in production, strong unique secrets, secure database credentials and correctly restricted cross-origin access. Do not publish environment files, passwords or API secrets in the repository.

Test account registration, login, password reset, role permissions, photo access, input validation, rate limits, session expiry and backup/restore procedures.

Agree on a retention and deletion policy with management. The repository describes an automated complaint-retention process; its exact rules should be reviewed against residence policy before production use.

Provide residents with a short privacy notice explaining what data is collected, who can access it, why it is used and whom to contact about concerns. Follow applicable South African privacy requirements, including POPIA where applicable.

10. Implementation Plan

Phase 1 — Review and requirements confirmation: demonstrate the current website and dashboard to management; confirm reporting categories, priority definitions, status names, room structure, staff responsibilities and privacy requirements.

Phase 2 — Complete integration: verify API/database configuration, resolve any outstanding defects, finish the mobile app's remaining data-service migration where mobile rollout is required, and remove development-only configuration from production.

Phase 3 — Quality and security testing: test common resident and management journeys, access permissions, photo uploads, mobile responsiveness, error handling, accessibility, performance and recovery procedures.

Phase 4 — Controlled pilot: invite a small, agreed group of residents and authorised staff to use the system for a limited period. Keep a fallback reporting route available during the pilot.

Phase 5 — Review and improve: gather feedback, measure the pilot, prioritise defects and agree on support and maintenance arrangements.

Phase 6 — Wider rollout: launch to the wider residence only after management approves the pilot results, operating process, privacy controls and ongoing ownership.

11. Success Measures

Number and percentage of maintenance reports submitted through the system during the pilot.

Percentage of reports with complete location and issue details.

Time between report submission and management acknowledgement.

Time spent by management consolidating or searching for maintenance reports.

Number of open, overdue and completed reports, reviewed alongside issue severity and staffing context.

Resident and staff feedback on ease of use, clarity of updates and reliability.

System availability, failed submissions, security incidents and support requests.

12. Risks and Mitigations

Connectivity or hosting outage: maintain a documented fallback reporting method and establish a recovery contact.

Incomplete mobile integration: treat the website as the pilot channel until the mobile app passes end-to-end tests.

Incorrect or incomplete reports: use clear form labels, required fields and photo guidance.

Unauthorised access or data exposure: test role permissions, protect secrets, enforce HTTPS and limit access to resident data.

Reports not updated after submission: assign a named operational owner and agree on a regular review routine.

Low adoption: explain the process to residents, provide a short guide and make the reporting link easy to access.

13. Sustainability and Ongoing Support

The system will need an identified owner after the competition. Ongoing work may include dependency updates, security patches, monitoring, backups, restoring data when needed, responding to user questions and improving features based on feedback.

Before rollout, management and the developer should agree who pays for any hosting or domain costs, who controls the production accounts and source repository, how changes are approved, how backups are handled, and what support can realistically be provided. Hosting costs and service limits should be confirmed against the selected providers at deployment time rather than assumed.

14. Request to Residence Management

We request the opportunity to present a live demonstration and work with Corridor Hill Residence management to validate the reporting process, nominate pilot users and agree on the requirements for a controlled pilot.

Approval is requested for: (1) a requirements and workflow review with management; (2) permission to test with sample or approved pilot data; (3) a designated management contact for maintenance workflow decisions; and (4) a review meeting at the end of the pilot to decide whether to proceed with wider implementation.

15. Conclusion

The Corridor Hill Residence Maintenance System offers a practical way to move maintenance reporting from scattered or paper-based communication into a structured digital workflow. Its resident reporting, complaint tracking and management dashboard are designed to improve visibility, record keeping and communication. With the remaining integration, deployment, testing and governance tasks completed, a controlled pilot can help determine how effectively the system supports the residence and what changes are needed before wider adoption.

Appendix A — Project components identified in the ZIP

- Website pages include the landing page, login, registration, room selection, resident dashboard, complaintsubmission, tracking and management dashboard.
- Frontend code is organised in HTML, CSS and JavaScript files under css/ and js/.
- The README describes an Express API and PostgreSQL database, with JWT authentication andresident/management role checks.
- The mobile/ folder contains an Expo/React Native app with authentication, resident screens and managementscreens.
- The README says some mobile data services still use Firebase while migration to REST API endpoints is underway.
- Deployment configuration is included for a GitHub Pages website and a Render-hosted API/database; actualproduction deployment must be confirmed.